Bandit - Level 04

Goal

The password for the next level is stored in the only human-readable file among ten, inside `inhere`, in the home of `bandit4`.

Connection

```
ssh -p 2220 bandit4@bandit.labs.overthewire.org
```

Use the password retrieved at the end of Bandit - Level 03.

Concepts

The filename mechanics are the same as Bandit - Level 01 — these files start with `-`, solved the same way with `./`. The new piece is different: distinguishing *binary* from *plain-text* content, not just getting a command to accept a filename. Full breakdown: Linux - File Type Detection.

Solution Approach

```
file ./-file*          # check the type of all ten files in one shot
# ./-file07: ASCII text   ← the only one that isn't "data"

cat ./-file07
```

Points of Friction

- First tried `cat ./-file04` directly — it ran without error (the `./` already solved the filename issue from Level 1), but printed garbled binary output instead of a password. The mistake wasn't syntax this time, it was assuming any file that *opens* must be the right one.
- `file -file0` (without `./`) hit the exact same leading-dash trap as Level 1 — `file` tried to parse `-file0` as options and errored. Confirms the Level 1 lesson generalizes to every command, not just `cat`.
- Checked files one at a time before realizing `file` accepts a wildcard and reports on all matches in a single line each — much faster than ten separate manual checks.

Key Takeaway

A command succeeding (no error, prints *something*) isn't the same as it succeeding with the *right* file. Binary noise is still valid output as far as the shell is concerned — `file` (or just eyeballing the result) is what actually confirms whether content is meant to be read by a human.

Next

`ssh -p 2220 bandit5@bandit.labs.overthewire.org` using the password found above.
